

Group 5

Full Name	Student ID
Joost Leliveld	1609726
Vic Brien	2049961
Ralph Dubbeldam	1460617
Tijn Bergmans	1245869
Jelco Claessens	1638173
Wouter Witteman	1379534



Eindhoven, June 18, 2026

Contents

1	Introduction	2
2	High-level Description of the Proposed Function (Proposal Phase)	2
3	Requirements (proposal phase)	3
4	Proposed approach	4
4.1	Theoretical Method	4
4.2	Functional Sub-systems	4
5	Sub-system implementation and validation	5
6	Vision Detection and Recognition	5
7	Person identification	5
8	Tracking and Following	6
9	Obstacle Detection and TOF sensor	7
9.1	Obstacle Avoidance Node	7
10	Motion Control Subsystem and state management	8
11	Integration and verification	9
11.1	Integrated Deployment	9
11.2	Requirements Verification	10
12	References	11

1 | Introduction

This report explains a proposed and integrated functional architecture of James Jetson; a robot designed to identify and track a target within a crowd of people. The details of the implementation are discussed in the chapters below.

The link to the github repository is: [Github Link](#)

The link of the final demonstration video is: [YouTube Link](#)

2 | High-level Description of the Proposed Function (Proposal Phase)

The robot will initialize in search-mode. In this mode it will move around the environment to observe the surroundings. The robot will detect people that are in the camera's field of view. From the detected people it will identify a specific person using a reference image of that person. When it has identified the person it will start following that person around the environment. When it has lost the person for a given amount of time, it will go back to search-mode. Furthermore, the robot will use its TOF sensor to detect and find its way around objects that are in its way.

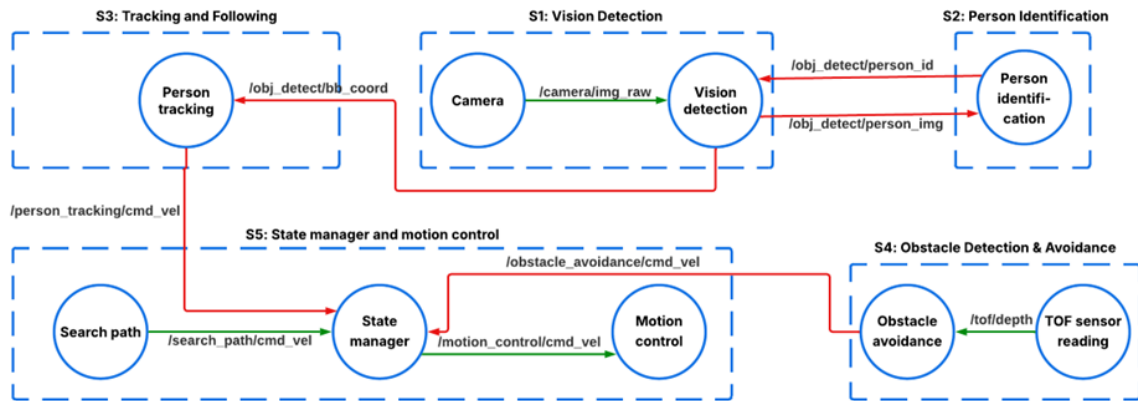


Figure 2.1: Proposed high-level ROS architecture, showing S0–S5 subsystems and topic flows.

3 | Requirements (proposal phase)

Based on the system description, the following requirements have been identified:

Table 3.1: System Requirements and Success Criteria

ID	Description	Success Criteria	Priority
R0	The robot shall detect all instances of people in the provided image.	Detect individuals within 8 meters in a group of 6 people, 1 meter apart, no full occlusions.	Mandatory
R1	The robot shall detect a pre-determined person based on a provided reference image.	90% accuracy in identifying the right person in a group of 6 within 8 meters.	Mandatory
R2	The robot shall follow the identified person in a flat open space.	Successful following in 90% of test cases for 1 minute (target speed < 5 km/h).	Mandatory
R3	The robot shall detect and avoid static obstacles shorter than 1 meter.	Avoid 90% of obstacles in test runs.	Mandatory
R4	The robot shall maintain a safe following distance.	Maintain 3-8 meters distance.	Mandatory
R5	System shall be tested indoors with quality lighting.	Perform within acceptable limits (no glare, reflections etc.).	Mandatory
R6	Robot shall issue an alert if target person is lost.	Alert within 2 seconds of losing visual contact.	Optional
R7	Robot shall operate for at least 20 minutes on battery.	Maintain functionality for 20 minutes.	Mandatory
R8	Log robot movements and obstacle detections.	Generate a log file with timestamps, events, and positions.	Optional

In Chapter 2 we describe the Person Identification node (S2), which subscribes to cropped person images and publishes a unique ID (or “None”) for each detection. Although our current implementation uses a random-ID stub, this module can be extended to run a deep-feature re-identification network.

4 | Proposed approach

4.1 | Theoretical Method

The robot uses a camera and applies neural networks to detect people in the frame. Detected person images are compared to a pre-defined reference image using a person identification neural network. Motors are controlled to follow the identified person and avoid obstacles using a TOF sensor for distance measurement.

4.2 | Functional Sub-systems

Table 4.1: Functional Subsystems and Components

Subsystem	Description / Components	Responsible
S1. Vision Detection & Recognition	Captures frames, detects persons, sends cropped images and bounding boxes to S2 and S3.	Tijn
S2. Person Identification	Compares cropped images to a reference using a person re-ID model (e.g., Siamese Network).	Wouter
S3. Tracking & Following	Receives coordinates from S1, uses a tracking algorithm (e.g., Kalman Filter), and generates movement commands.	Joost, Vic
S4. Obstacle Detection	Uses TOF sensor for obstacle detection, triggers avoidance behavior via S5.	Ralph
S5. Motion Control & State Management	Manages robot states (following, avoiding, searching), sends motor commands.	Jelco, Ralph

5 | Sub-system implementation and validation

A state flow diagram of the functional architecture can be seen in Figure 5.1 below. In the following sections, each sub-system is described in more detail.

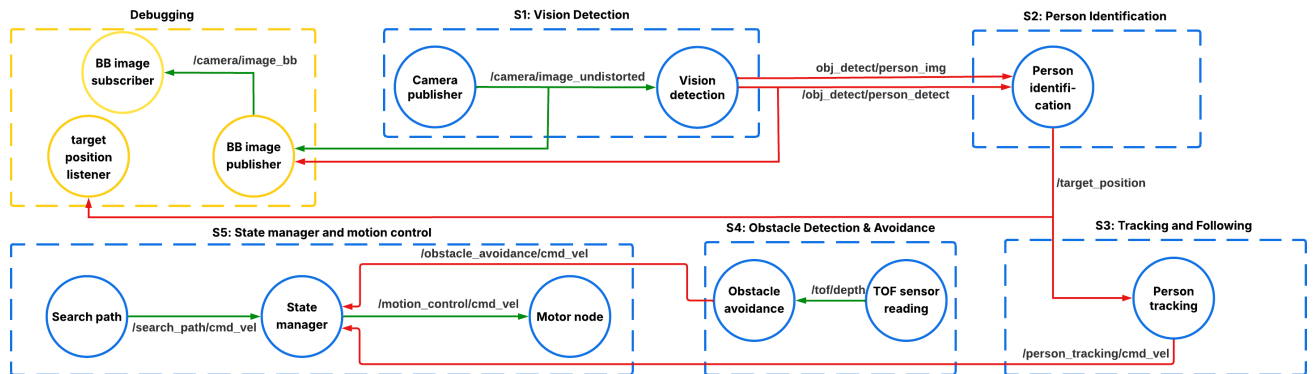


Figure 5.1: Implemented high-level ROS architecture, showing S0-S5 and debugging nodes with topic flows.

6 | Vision Detection and Recognition

The vision detection node audits the camera stream published by the camera node in order to detect all people present in the frame. Each frame captured by the node `camera_publisher_node` and published to the `/camera/image_raw` topic is passed through the Mobilenet SSD model for object detection. All detections are analyzed. When the model detects an instance of the class "person" with a confidence score of 60% or larger, a message is published to the `/obj_detect` node. This message contains the coordinates of the detected person in the camera frame and a crop of the image region within the bounding box. This image containing the person will be used for identification purposes.

Initially, the Mobilenet SSD model was implemented using the OpenCV library, using `.caffe` and `.prototxt` configuration files to construct an OpenCV DNN module. Though providing a useful proof of concept, during testing of the full system pipeline this approach proved too slow for a real time detection and tracking implementation, with a latency of approximately $330ms$ per forward pass through the model. After initial attempts to decrease the latency by running the OpenCV DNN module on the GPU instead of the CPU proved unfruitful, the module was replaced by a TensorRT module, optimized for running on the Jetson GPU. Additionally, the Mobilenet SSD model was replaced by the PedNet model from the jetson-inference library [1], specifically designed for detection of people.

These changes decreased the latency per forward pass to approximately $100ms$, which proved low enough for real-time detection and tracking functionality.

7 | Person identification

The person-identification node runs in parallel with the person-detection node and is responsible for detecting the target person in all the snippets of persons it receives from the person detection node. It listens on the topic `/obj_detect/person_detect` where the detector sends a JPEG crop of every person it finds together with the crop's position in the original image. As soon as the node starts, it scans a folder that contains several photos of the target person. Each photo is passed once through a pre-trained person identification network (OSNet-x0.5) that has been converted to a TensorRT FP16 engine (optimized to run on the Jetson Nano). The network produces a 512-dimensional feature vector for every photo. These vectors are normalised, averaged and stored as a single template that represents the person of interest. The person identification node sends every new crop that it receives through the TensorRT engine, resulting in another 512-dimensional vector that represents the person in the snippet. It then computes the cosine similarity between this vector and the stored template. One inference takes roughly sixty milliseconds on the Jetson, so the node can easily keep up with multiple frames per second. If the similarity score exceeds

a predefined threshold, the bounding box of the person is published to the `/target_position` topic at a rate of 5 Hz. In the case of no match, no bounding box is published. This is interpreted by the state manager to switch to the proper robot state.

8 | Tracking and Following

The Person Tracking node is responsible for following the person that was identified by S2. This is done by maintaining the position of the person at the center of the camera's field of view. Using bounding box coordinates received from the Person Detection node the tracking algorithm computes the offset between the bounding box center and the frame center. From this offset the angular speed is determined. The forward speed of the robot is adaptively controlled based on both the height and the horizontal offset of the bounding box. The height relative to the height of the image indirectly represents the distance to the person. The robot reduces its forward velocity when the person is off-center to avoid lateral drift, where driving forward would cause the target to leave the camera view. No explicit depth estimation is necessary, as the adaptive feedback loop inherently manages variations in distance. This dual feedback mechanism helps the robot maintain both lateral alignment and longitudinal distance with the tracked person in real-time, without requiring depth sensors.

The calculated velocity commands from this node are published on the topic `/person_tracking/cmd_vel`. These commands are then evaluated by the State Manager node. In scenarios where the target is temporarily lost (i.e., no valid bounding box is received), the node publishes speed commands of 0. This way the state manager node will know to listen to another topic. Since it could be that the bounding box was temporarily not found for a few frames and we don't want to switch states continuously.

9 | Obstacle Detection and TOF sensor

A crucial module of the robot's sensor body is the Time-of-Flight (ToF) Sensor Node which reads distance values from the surroundings. This sensor is used to trigger the obstacle avoidance maneuver.

9.1 | Obstacle Avoidance Node

The Obstacle Avoidance Node creates collision avoidance using the measurements of the proximity of the environment in the Time-of-Flight (ToF) sensor. The node subscribes to the `/tof/depth` topic to obtain `sensor_msgs/Range` messages and issues velocity commands (`geometry_msgs/Twist`) in the `/obstacle_avoidance/cmd_vel` topic.

9.1.1 | Design Overview

The node operates as a finite state machine (FSM) comprising four states:

- **IDLE**: Awaits incoming range data and monitors for obstacles.
- **TURN_LEFT**: Initiates a leftward turn upon detecting an obstacle.
- **FORWARD**: Commands the robot to move forward after turning.
- **TURN_RIGHT**: Realigns the robot to its initial heading.

State transitions are managed through timed callbacks, with appropriate velocity commands issued for each maneuver.

9.1.2 | Obstacle Detection

A sliding window filter with the size `window_size`, which keeps the latest range readings within buffer. If a configurable percentage that is spread by `detection_ratio` of those readings are farther away than the `safe_distance` parameter, the avoidance is activated. This method offers stability and it filters out sensor noises with temporary periods and maintains a high level of reactivity.

9.1.3 | Movements

To make the movements more robust, encoders are used. These keep track on how much the motors have moved. For each movement a target is set on how much the wheels should turn to go forward or turn. When the target is met, the movement is completed.

9.1.4 | Obstacle Avoidance Maneuver

When an obstacle is detected, the robot performs a sequence: a left turn, forward motion, and a right turn, followed by stopping. This reactive strategy, based on fixed velocity commands, allows simple obstacle circumvention without position feedback. Parameters such as speed and turning distance are configurable via ROS. When the bot still has not reached the end of the obstacle it will repeat this maneuver.

10 | Motion Control Subsystem and state management

The robot's motion control is controlled by a central *state manager node* which is responsible for deciding what kind of motion command that should be dispatched to the robot at each instance. It subscribes three different behavior modules with their own `cmd_vel` topics and forwards the command with the highest priority to the motion controller.

The three behavior components are:

- **Obstacle avoidance:** continuously checks the presence of close obstacles and stops, or avoids them whenever necessary. Safety is of paramount importance to do this, and this is a priority.
- **Follow (Person Tracking):** Have the robot follow a detected person, when there are no obstacles.
- **Searching(/Exploration):** Commands the robot to explore, if not a person is observed and no obstacle is nearby.

The state manager implements a pretty basic priority system: obstacle avoidance overrides person following, which overrides searching.

11 | Integration and verification

11.1 | Integrated Deployment

The deployment integrates the following functional subsystems:

- **Vision Detection & Recognition (S1):** Captures images from the onboard camera, detects people using a TensorRT-optimized PedNet neural network, and publishes both bounding box coordinates and cropped person images via the `/obj.detect` topic.
- **Person Identification (S2):** Subscribes to cropped person images from S1 and compares them to a provided reference image using a re-identification model. It publishes identification results for each detection via a dedicated topic.
- **Tracking & Following (S3):** Receives bounding box data from S1 and computes tracking commands to keep the identified person centered in the robot's view. These commands are published on the `/person_tracking/cmd_vel` topic.
- **Obstacle Detection (S4):** Processes range data from a ToF sensor, identifies nearby obstacles using a sliding window filter, and triggers avoidance behavior as needed.
- **Obstacle Avoidance Node:** Subscribes to ToF data and publishes reactive motion commands via the `/obstacle_avoidance/cmd_vel` topic when obstacles are detected.
- **Motion Control & State Management (S5):** Acts as a central decision-making module that subscribes to the motion command topics from the other subsystems. It uses a hierarchical priority system to determine which command to forward to the robot's motor controllers:
 1. Obstacle avoidance commands
 2. Person tracking commands
 3. Searching (exploration) commands

11.2 | Requirements Verification

Table 11.1: Requirements Verification Results

ID	Satisfied	Remark with Specific Details
R0	Yes	The robot could detect the people even further than 8 meters. More than 15 meters was achieved.
R1	Yes	When the person of interest wore different clothing the bot could differentiate with high certainty who the person of interest was.
R2	Yes	It followed the person in almost all cases. Except when the person was running out of the camera field. Or when the person gets closer than 1 meter.
R3	Yes	It can evade objects smaller than 1.5 meters. When the objects become longer, the bot will slowly drift away or towards the obstacle when evading.
R4	Yes	The following distance of around 5 meters was met at all times for a person that is walking under 5 km/h.
R5	Yes	No issues were found and worked as expected.
R6	No	As this was an optional requirement, this was not made due to limited time.
R7	Yes	We used another battery with an capacity of 100 Wh and could keep the bot running for 10 hours. This means that the bot used 10 Watts of power on average.
R8	Yes	These log files temporary generated for debugging. The final code does not include these.

12 | References

- [1] dusty-nv. jetson-inference: Hello AI World guide to deploying deep-learning inference networks and deep vision primitives with TensorRT and NVIDIA Jetson. <https://github.com/dusty-nv/jetson-inference>, 2016-. Accessed: 2025-06-15.